COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Sydney pursuant to Part VB of the Copyright Act 1968 (**the Act**). The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

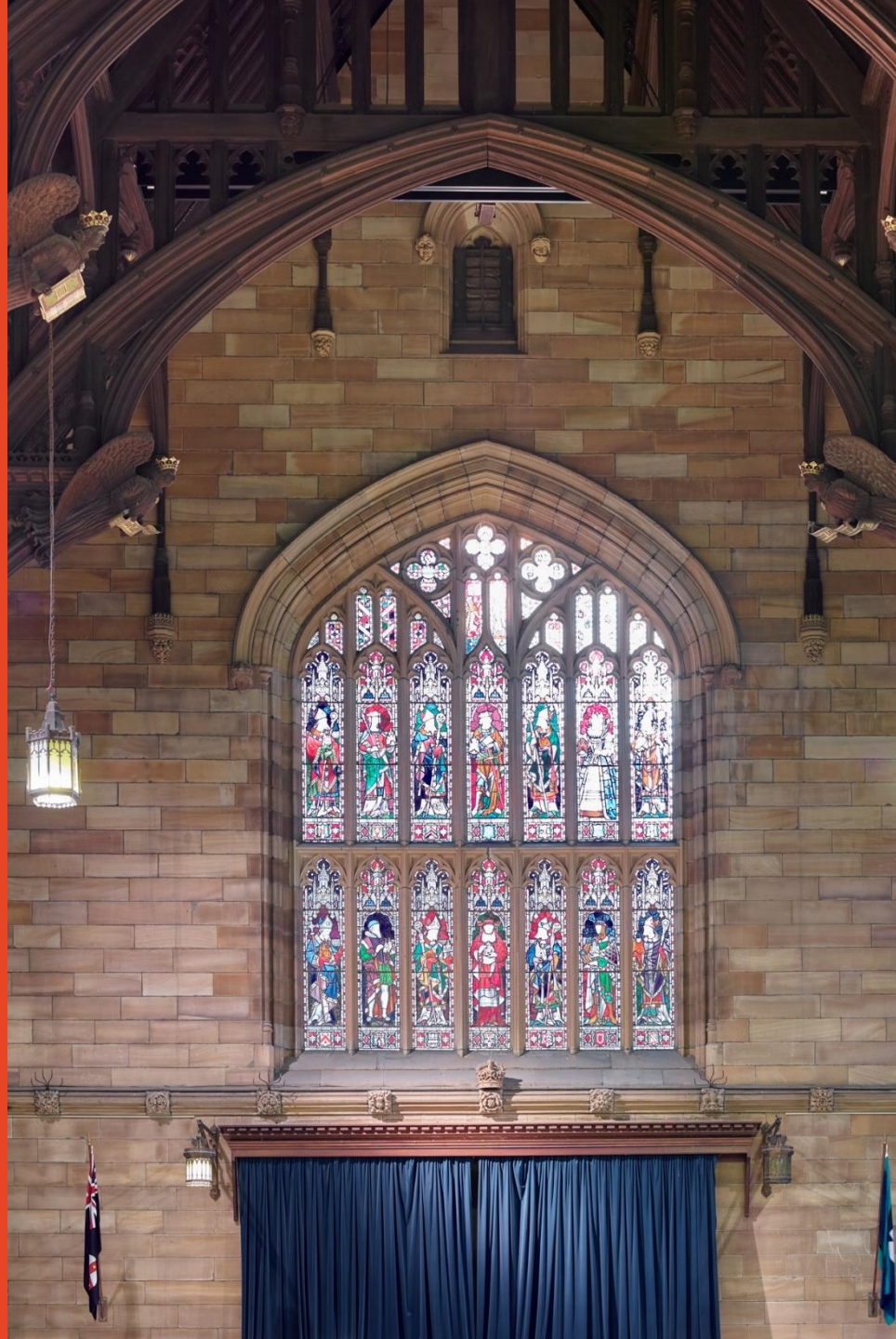
Do not remove this notice.

COMP2123
Data structures and Algorithms
Lecture 10: Divide and Conquer
[GT 3.1 and 8]

Sri AravindaKrishnan Thyagarajan (Aravind)
School of Computer Science

Some content is taken from material provided by the textbook publisher Wiley.

THE UNIVERSITY OF SYDNEY



Updates

Assignment 2 is due soon

Replacement exam for IST (mid-sem exam) has been announced on Ed and via email

Our in-semester survey is back!



Divide and Conquer

Divide and Conquer algorithms can normally be broken into these three parts:

1. Divide If it is a base case, solve directly, otherwise break up the problem into several parts.

2. Recur/Delegate Recursively solve each part [each sub-problem].

3. Conquer Combine the solutions of each part into the overall solution.

Divide and Conquer

1. Divide If it is a base case, solve directly, otherwise break up the problem into several parts.

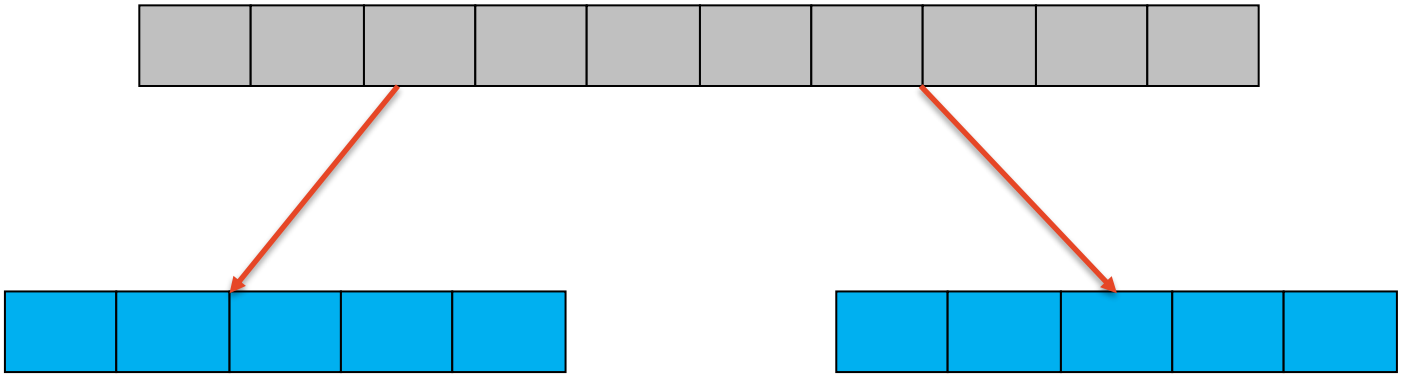
Typical base case:

Subproblem of constant size (usually 0 or 1 elements) for which you can compute the solution explicitly



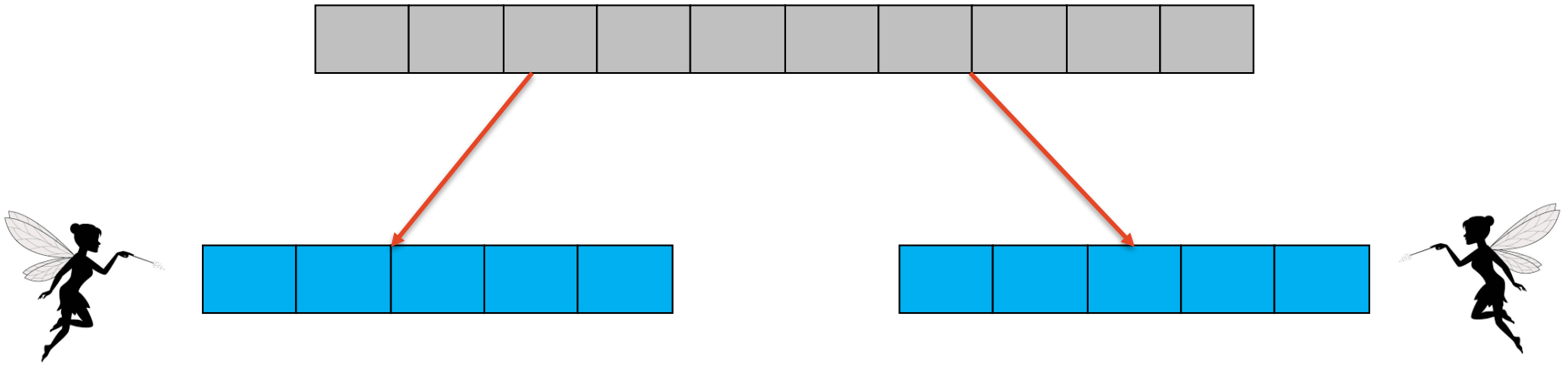
Divide and Conquer

1. Divide If it is a base case, solve directly, otherwise break up the problem into several parts.



Divide and Conquer

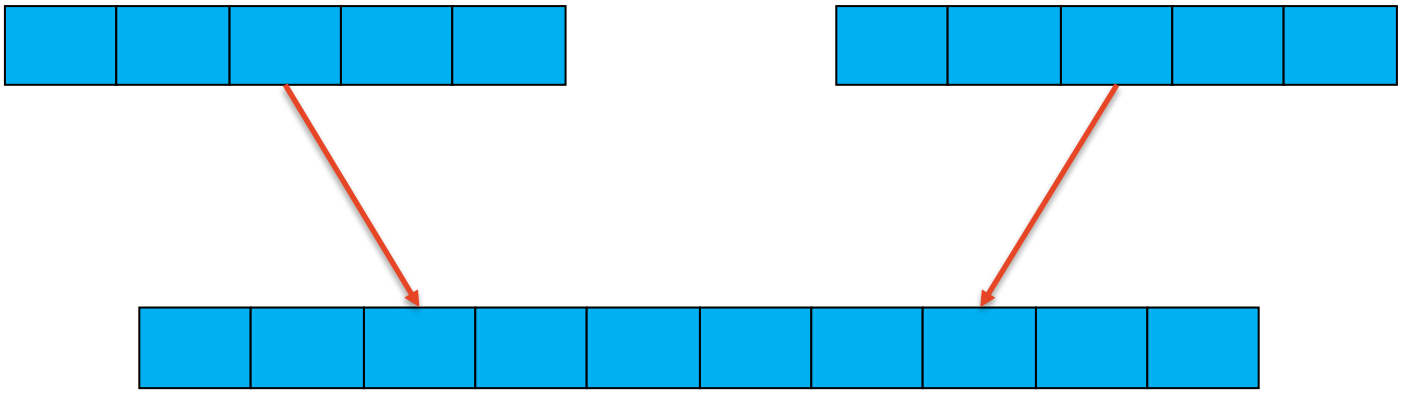
2. Recur/Delegate Recursively solve each part [each sub-problem].



The sub-problems are solved by the Recursion Fairy (similar to induction hypothesis), so we don't have to worry about them.

Divide and Conquer

3. Conquer Combine the solutions of each part into the overall solution.



Searching Sorted Array

Given A sorted sequence S of n numbers $a_0, a_1, \ldots, a_{n-1}$ stored in an array A[0, 1, …, n - 1].

Problem Given a number x, is x in S?

0	2	5	7	12	22	25	37	39	50	55	80

Searching: Naïve Approach

Problem Given a number x, is x in S?

Idea Check every element in turn to see if it is equal to x.

```
for e in S do
    if e equals x then
        return "Yes"
return "No"
```

Found an element equal to x in S

There was no element equal to x in S

0	2	5	7	12	22	25	37	39	50	55	80

Running Time O(n)

Binary Search in sorted A[0 to n-1]

1. If the array is empty, then return "No"
2. Compare x to the middle element, namely A[⌊n/2⌋]
3. If this middle element is x, then return "Yes"
4. When the middle element is not x: if A[⌊n/2⌋]>x, then recursively search A[0 to ⌊n/2⌋-1]
5. if A[⌊n/2⌋]<x, then recursively search A[⌊n/2⌋+1 to n-1]

0	2	5	7	12	22	25	37	39	50	55	80

Binary Search Pseudocode

```
def binary_search(A, left, right, x):
    # A is sorted and left <= right
    # looking for x in A[left:right]

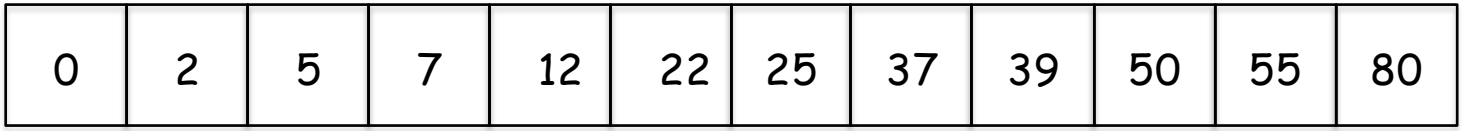
    if left > right then
        return “unsuccessful”

    mid = floor((left + right) / 2)
    if A[mid] < x then
        return binary_search(A, mid + 1, right, x)
    else if A[mid] > x then
        return binary_search(A, left, mid-1, x)
    else
        return mid
```

Heads up: pseudocode textbook uses indexing from 1 to n, not 0 to n-1

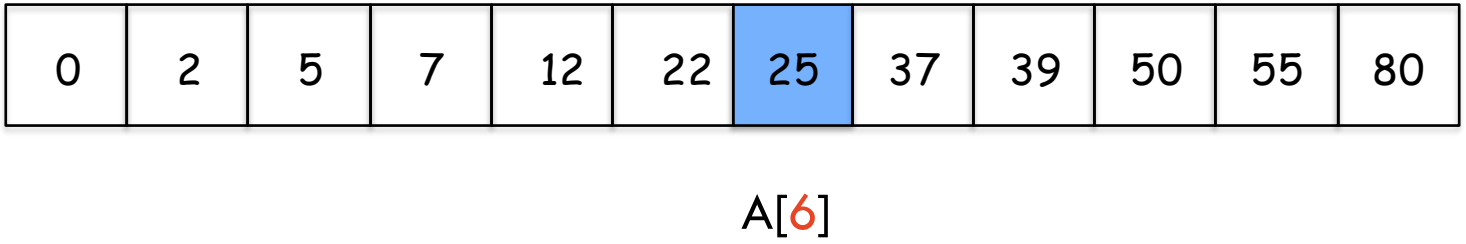
Binary Search

- Example, search for x=5



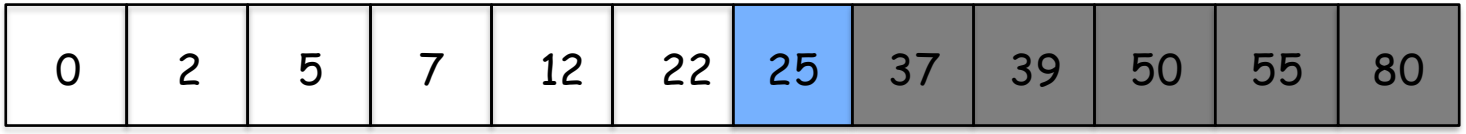
Binary Search

- Example, search for x=5



Binary Search

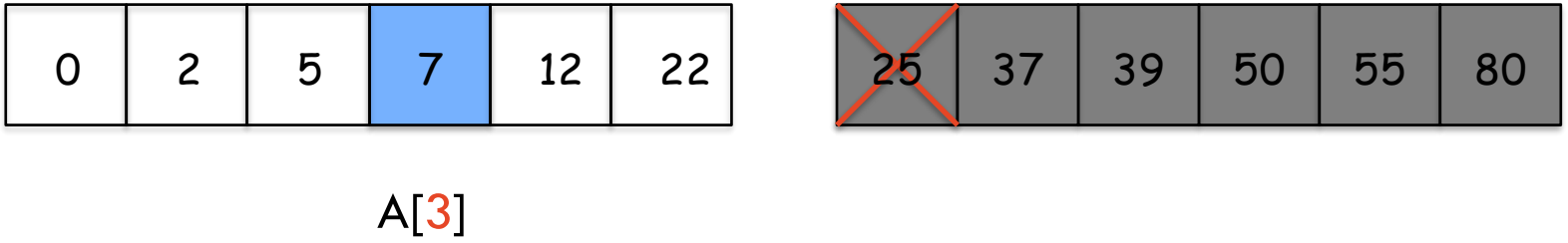
- Example, search for x=5



$A[6] = 25 > 5 = x$

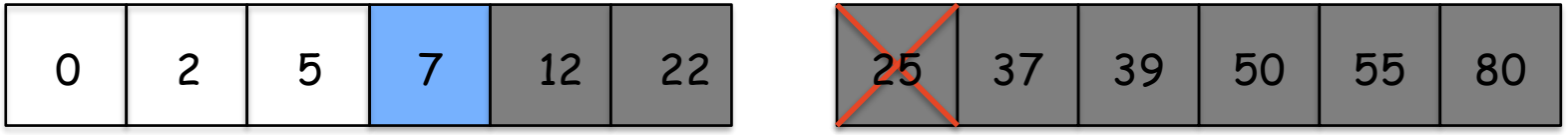
Binary Search

– Example, search for x=5



Binary Search

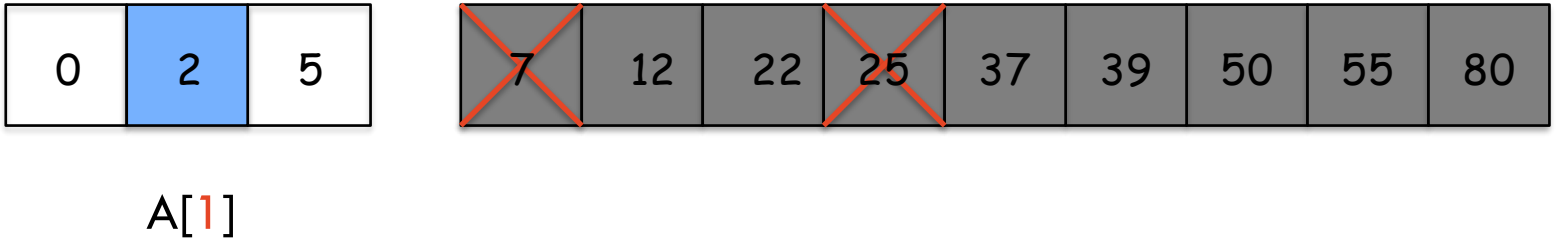
- Example, search for x=5



A[3] $= 7 > 5 = x$

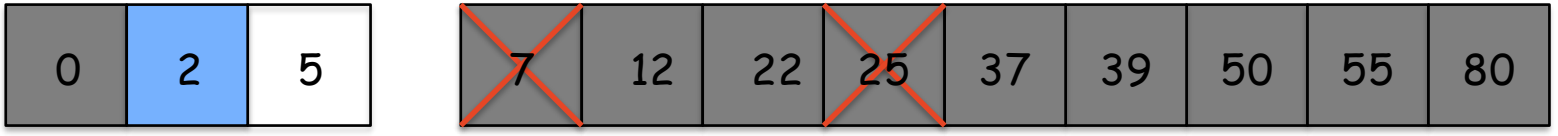
Binary Search

– Example, search for x=5



Binary Search

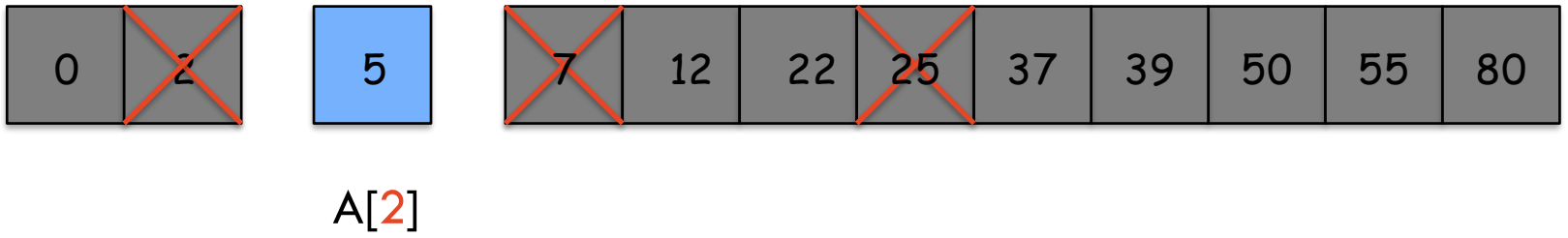
- Example, search for x=5



$A[1] = 2 < 5 = x$

Binary Search

– Example, search for x=5



Binary search correctness

Invariant: If x is in A before the divide step, then x is in A after the divide step

- if $A[\lfloor n/2 \rfloor] > x$, then x must be $A[0 \text{ to } \lfloor n/2 \rfloor - 1]$
- if $A[\lfloor n/2 \rfloor] < x$, then x must be in $A[\lfloor n/2 \rfloor + 1 \text{ to } n-1]$

Every divide step leads to a smaller array.

Thus, if x in A, we will eventually inspect its position due to the invariant and return "Yes".

Thus, if x in not in A, then eventually we reach the empty array and return "No".

Recurrence formula

An easy way to analyze the time complexity of a divide-and-conquer algorithm is to define and solve a recurrence

Let T(n) be the running time of the algorithm, we need to find out:

- Divide step cost in terms of n
- Recur step(s) cost in terms of T(smaller values)
- Conquer step cost in terms of n

Together with information about the base case, we can set up a recurrence for T(n) and then solve it.

$$T(n) = \begin{cases} \text{“Recur” + “Divide and Conquer”} & \text{for } n > 1 \\ \text{“Base case” (typically } O(1)) & \text{for } n = 1 \end{cases}$$

Binary search on an array complexity analysis

Divide step (find middle and compare to x) takes O(1)

Recur step (solve left or right subproblem) takes T(n/2)

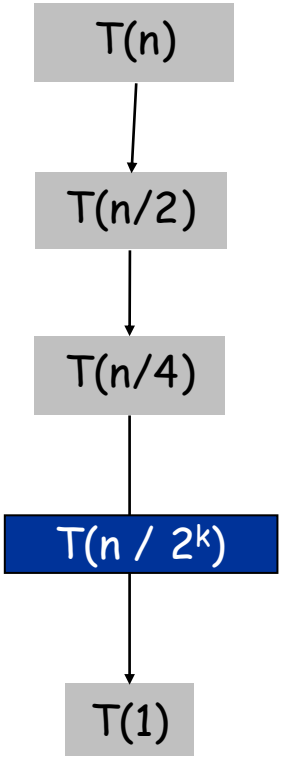
Conquer step (return answer from recursion) takes O(1)

Now we can set up the recurrence for T(n):

$$T(n) = \begin{cases} T(n/2) + O(1) & \text{for } n > 1 \\ O(1) & \text{for } n = 1 \end{cases}$$

This solves to $T(n) = O(\log n)$, since we can only halve the input O(log n) times before reaching a base case

Proof by unrolling: T(n) = T(n/2) + O(1)



1 (of size n)

1 (of size n/2)

1 (of size n/4)

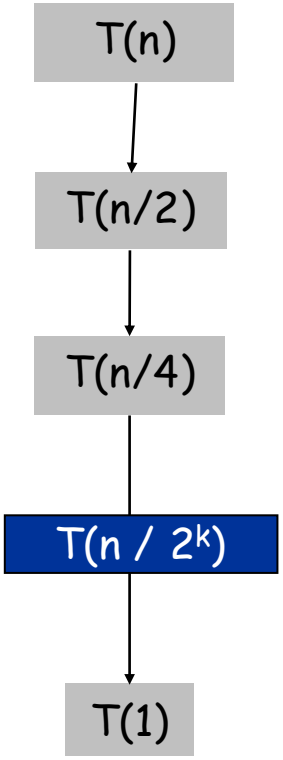
. . .

1 (of size $n/2^k$)

. . .

1 (of size 1)

Proof by unrolling: T(n) = T(n/2) + O(1)



1 (of size n) $\rightarrow$ c

1 (of size n/2) $\rightarrow$ c

1 (of size n/4) $\rightarrow$ c

$\log_2 n$

. . .

1 (of size $n/2^k$) $\rightarrow$ c

. . .

1 (of size 1) $\rightarrow$ c

$c \log_2 n$

Binary search on a linked list complexity analysis

Divide step (find middle and compare to x) takes O(n)

Recur step (solve left or right subproblem) takes T(n/2)

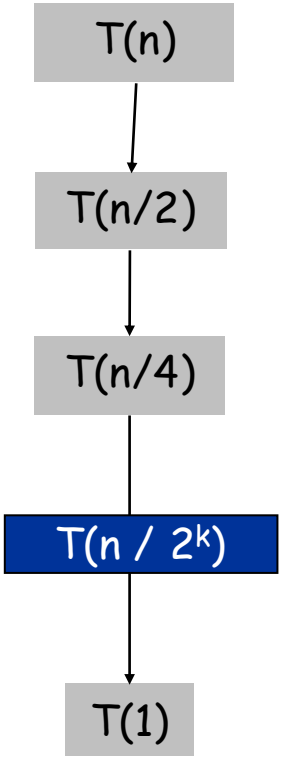
Conquer step (return answer from recursion) takes O(1)

Now we can set up the recurrence for T(n):

$$T(n) = \begin{cases} T(n/2) + O(n) & \text{for } n > 1 \\ O(1) & \text{for } n = 1 \end{cases}$$

This solves to $T(n) = O(n)$, since to access the next index we end up with $n/2 + n/4 + n/8 + \ldots$

Proof by unrolling: T(n) = T(n/2) + O(n)



1 (of size n)

1 (of size n/2)

1 (of size n/4)

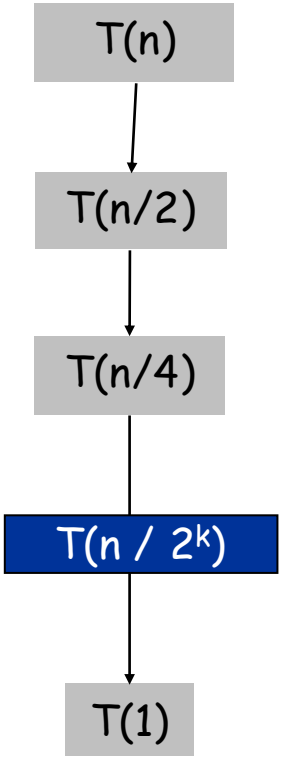
. . .

1 (of size $n/2^k$)

. . .

1 (of size 1)

Proof by unrolling: T(n) = T(n/2) + O(n)



$\log_2 n$

1 (of size n) $\rightarrow$ c n

1 (of size n/2) $\rightarrow$ c n/2

1 (of size n/4) $\rightarrow$ c n/4

. . .

1 (of size $n/2^k$) $\rightarrow$ c $n/2^k$

. . .

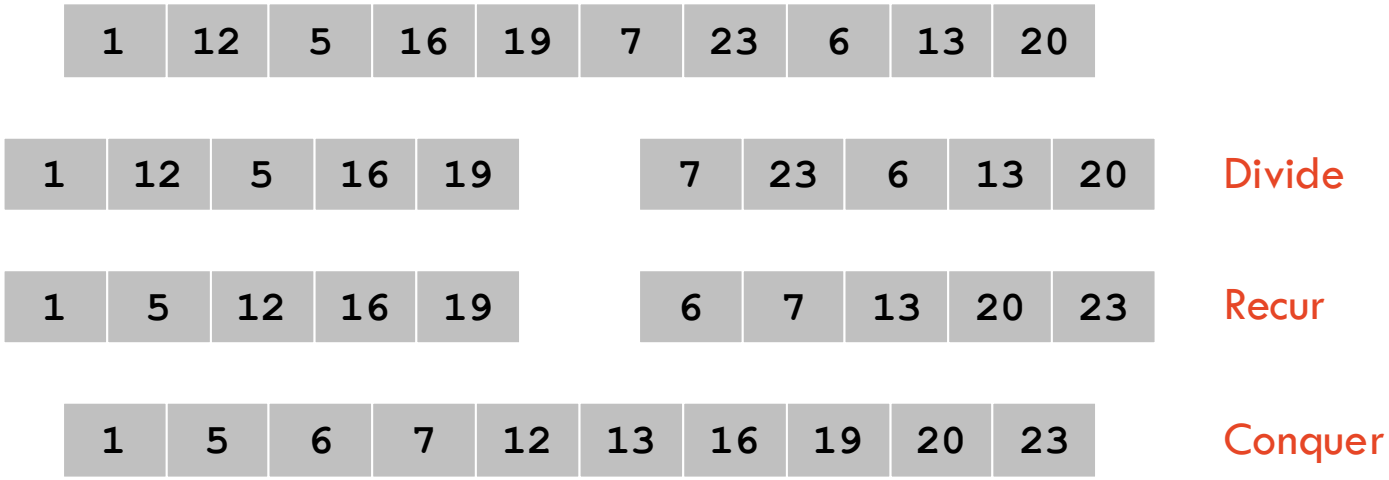
1 (of size 1) $\rightarrow$ c 1

---

2 c n

Merge-Sort

1. Divide the array into two halves.
2. Recur recursively sort each half.
3. Conquer two sorted halves to make a single sorted array.



Merge-Sort pseudocode

```
def merge_sort(S):
    # base case
    if |S| < 2 then
        return S

    # divide
    mid ← [|S|/2]
    left ← S[:mid]    # doesn't include S[mid]
    right ← S[mid:]   # includes S[mid]


    # recur
    sorted_left ← merge_sort(left)
    sorted_right ← merge_sort(right)

    # conquer
    return merge(sorted_left, sorted_right)
```

How?

Merge

Input Two sorted lists.

Output A new merged sorted list.

To merge, we use:

- O(n) comparisons.
- An array to store our results.



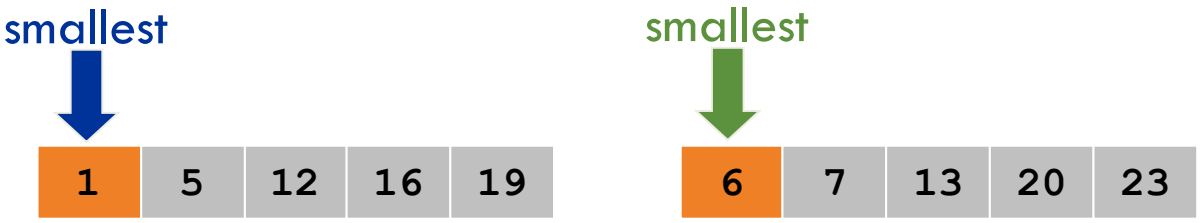
Result:



Merge

Merge Algorithm

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into the resultant array.
- Repeat until done.



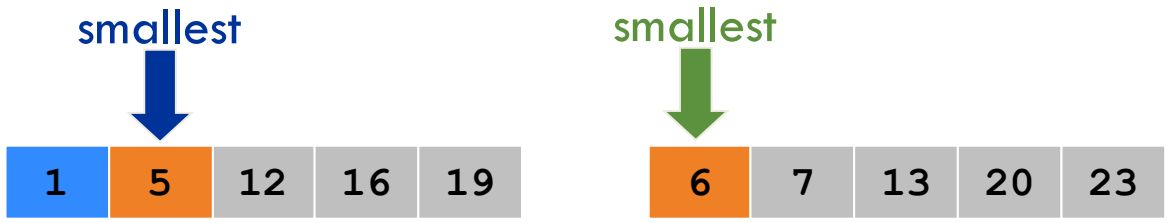
Result:



Merge

Merge Algorithm

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into the resultant array.
- Repeat until done.



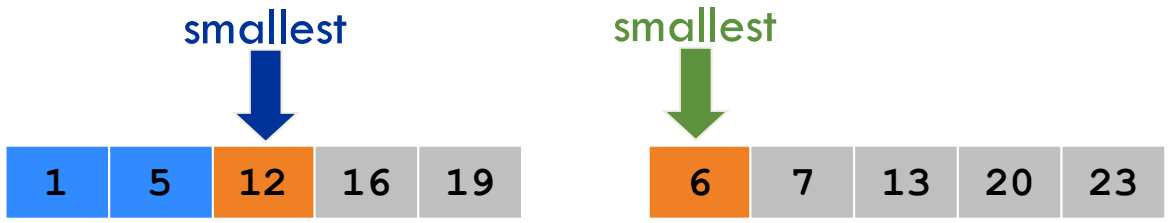
Result:



Merge

Merge Algorithm

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into the resultant array.
- Repeat until done.



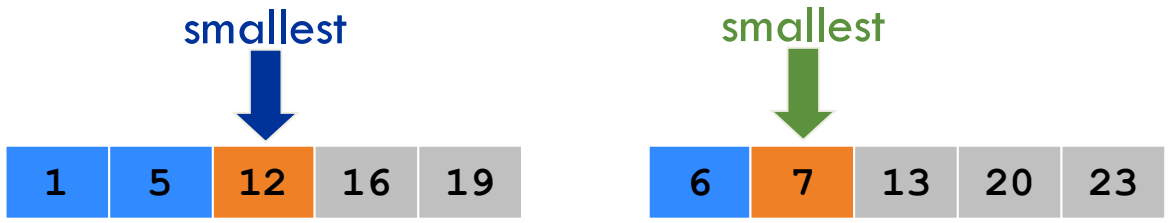
Result:



Merge

Merge Algorithm

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into the resultant array.
- Repeat until done.



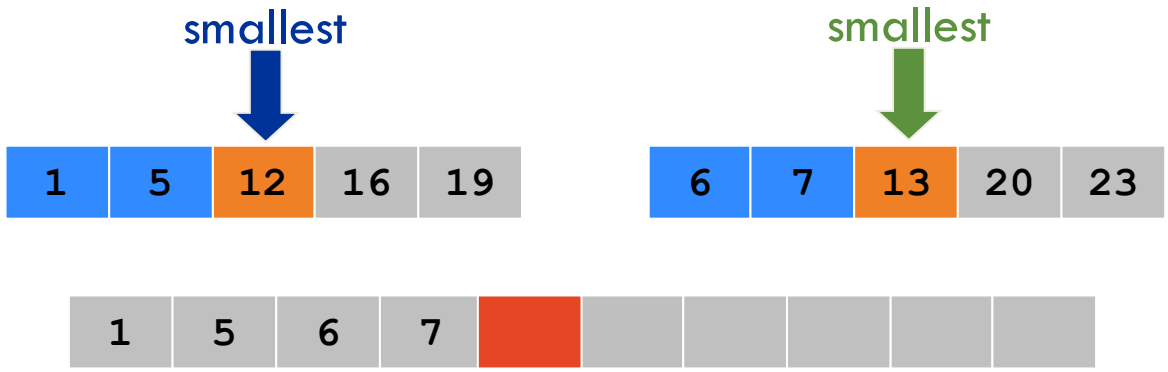
Result:



Merge

Merge Algorithm

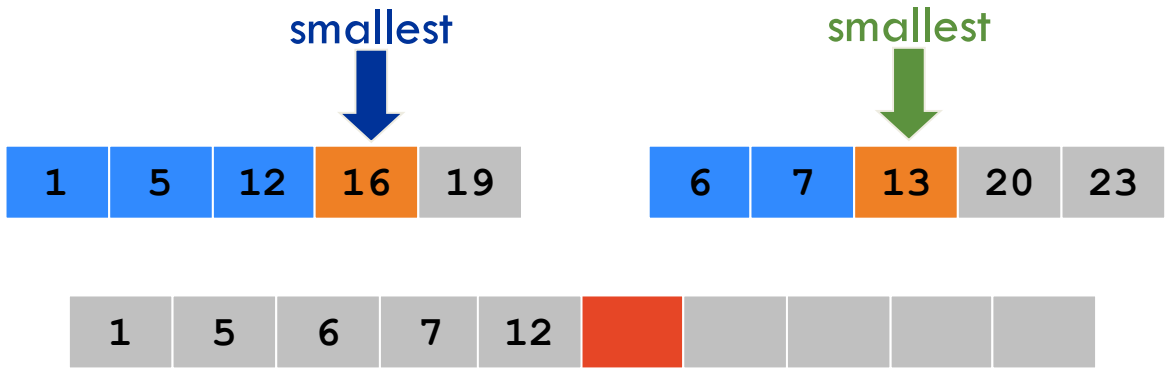
- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into the resultant array.
- Repeat until done.



Merge

Merge Algorithm

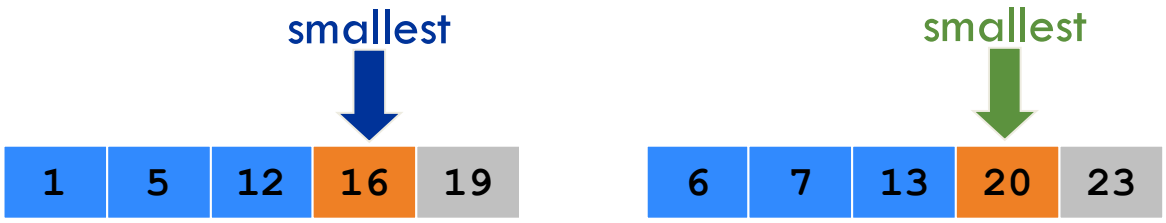
- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into the resultant array.
- Repeat until done.



Merge

Merge Algorithm

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into the resultant array.
- Repeat until done.



Result:



Merge

Merge Algorithm

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into the resultant array.
- Repeat until done.



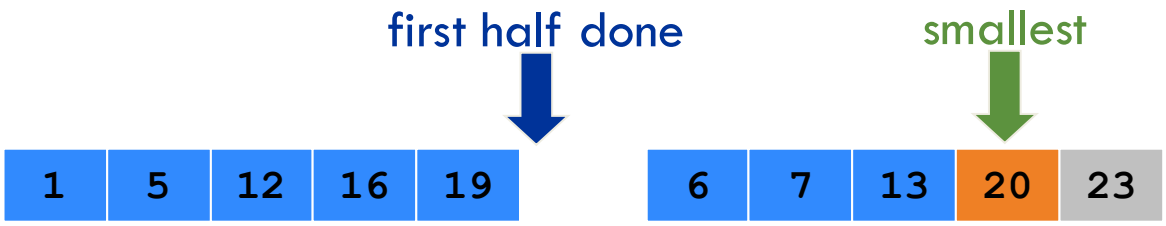
Result:



Merge

Merge Algorithm

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into the resultant array.
- Repeat until done.



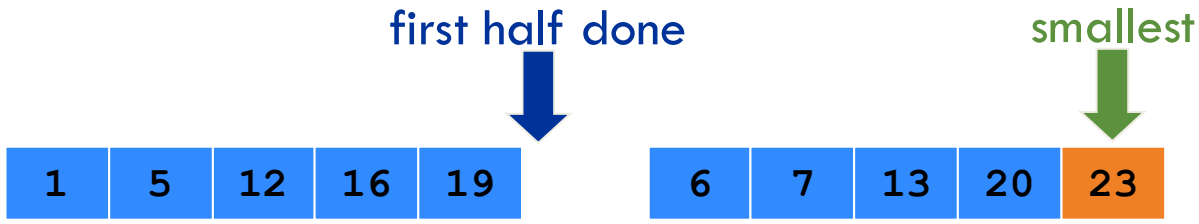
Result:



Merge

Merge Algorithm

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into the resultant array.
- Repeat until done.



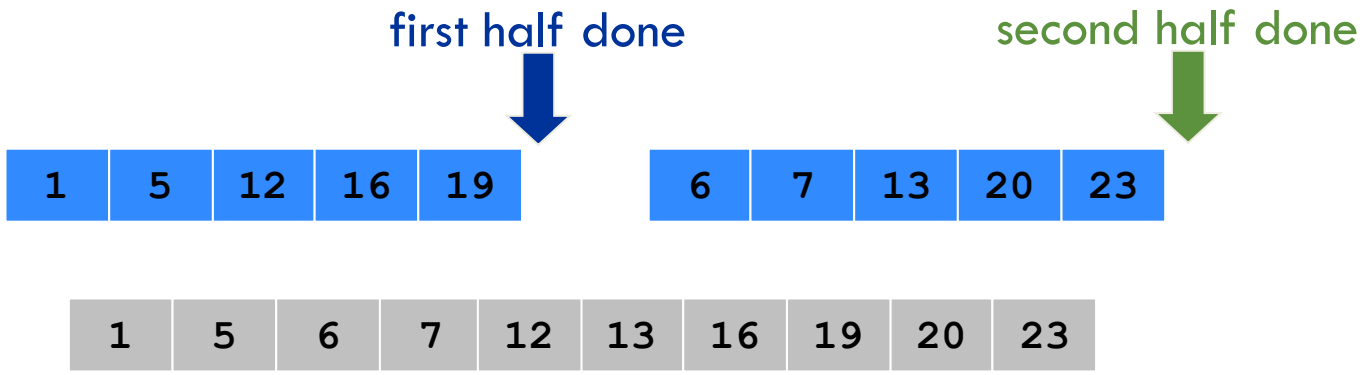
Result:



Merge

Merge Algorithm

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into the resultant array.
- Repeat until done.



Merge: Implementation

```
def merge(L, R):
    result ← array of length (|L| + |R|)
    l, r ← 0, 0
    while l + r < |result| do
        index ← l + r
        if r ≥ |R| or (l < |L| and L[l] < R[r]) then
            result[index] ← L[l]
            l ← l + 1
        else
            result[index] ← R[r]
            r ← r + 1
    return result
```

Merge: Correctness

Induction hypothesis:

- After the i-th iteration, our result contains the i smallest elements in sorted order

Base case:

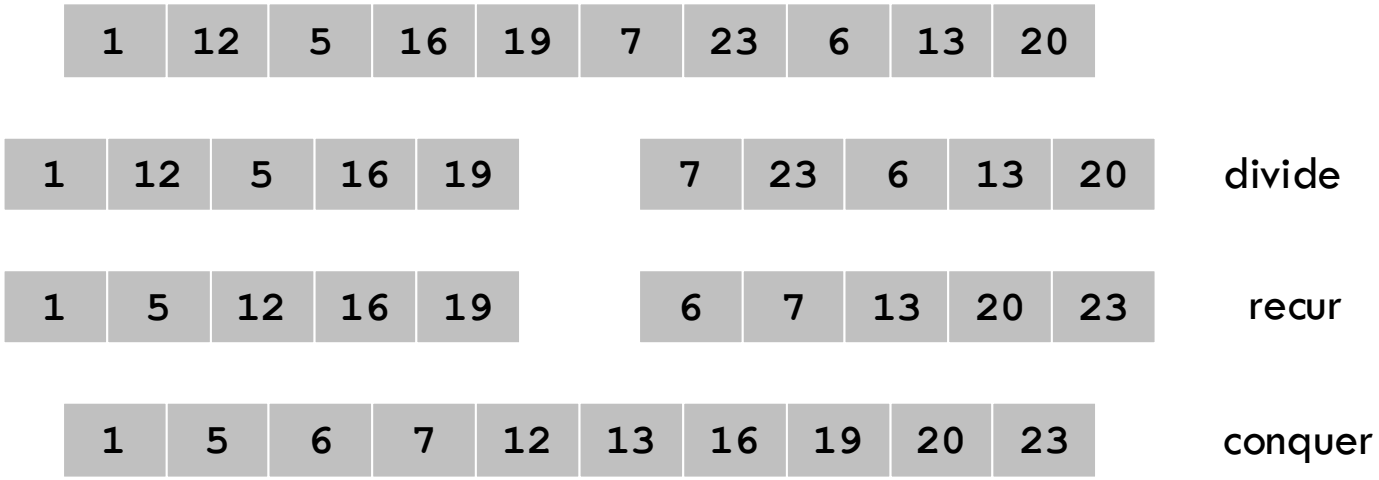
- After 0 iterations, our result is empty, so it contains the 0 smallest elements in sorted order

Induction:

- Assume IH after iteration k, to prove it after iteration k+1
- Since both halves are sorted and we add the smallest element not already in result, result now contains the k+1 smallest elements
- Sorted order follows from the fact that both halves are sorted, thus adding the smallest element implies sorted order of result

Merge-Sort

1. Divide array into two halves.
2. Recur Recursively sort each half.
3. Conquer Merge two sorted halves to make a sorted whole.



Merge-Sort: Correctness

Induction hypothesis:

- Merge-Sort correctly sorts an array of size i

Base case:

- If our array has size 0 or 1, it's already sorted

Induction:

- Assume IH for all arrays up to size k, to prove it for array of size k+1
- Splitting the array in half gives us two array of size at most k, so by IH those are sorted correctly
- We proved that given two sorted arrays, Merge returns a correctly sorted array containing the elements of both arrays
- Hence, by running Merge on the two sorted halves, we sort the original array

Merge sort complexity analysis

Divide step (find middle and split) takes $O(n)$

Recur step (solve left and right subproblem) takes 2 $T(n/2)$

Conquer step (merge subarrays) takes $O(n)$

Now we can set up the recurrence for $T(n)$:

$$T(n) = \begin{cases} 2\,T(n/2) + O(n) & \text{for } n > 1 \\ O(1) & \text{for } n = 1 \end{cases}$$

This solves to $T(n) = O(n \log n)$

Solving recurrences by unrolling

General strategy:

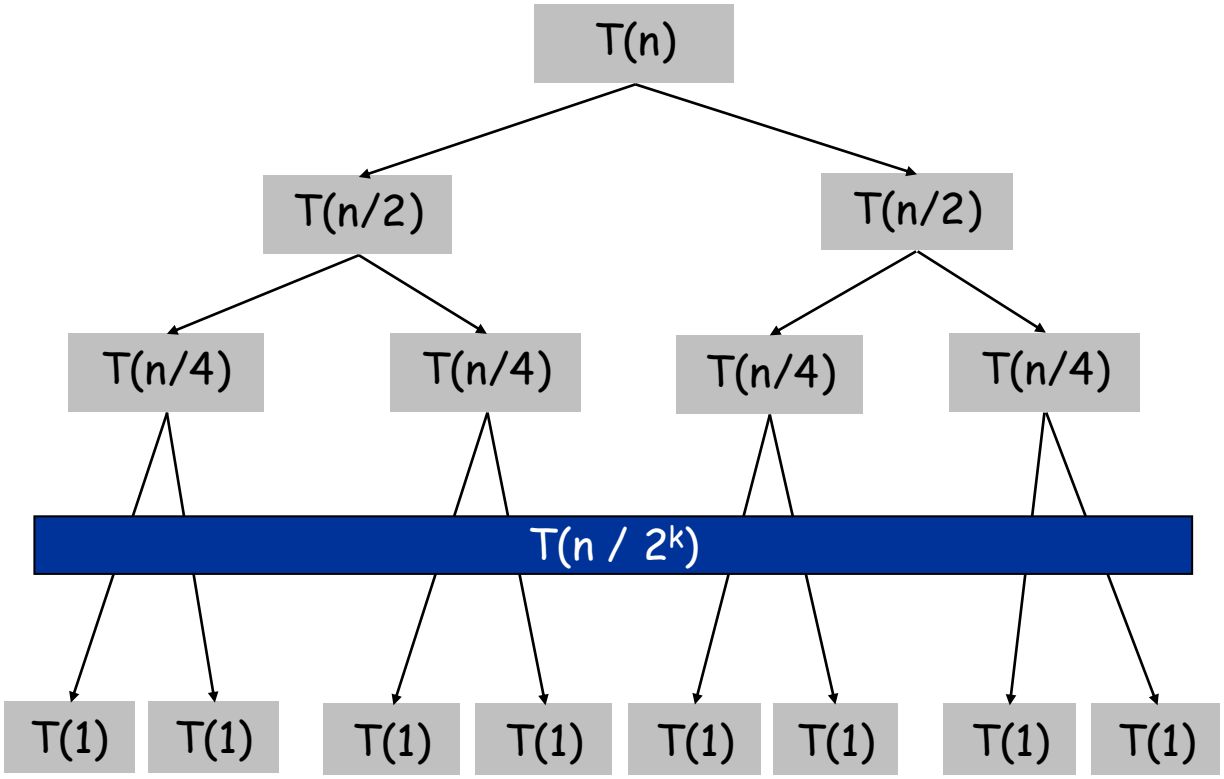
- Analyze first few levels
- Identify the pattern for a generic level
- Sum up over all levels

To verify the solution, we can substitute guess into the recurrence and prove it formally using induction

For Merge sort this method yields $T(n) = O(n \log n)$

There is a "Master theorem" (see textbook) that can handle most recurrences of interest, but unrolling is enough for our purposes

Proof by unrolling: T(n) = 2 T(n/2) + O(n)



1 (of size n)

2 (of size n/2)

4 (of size n/4)

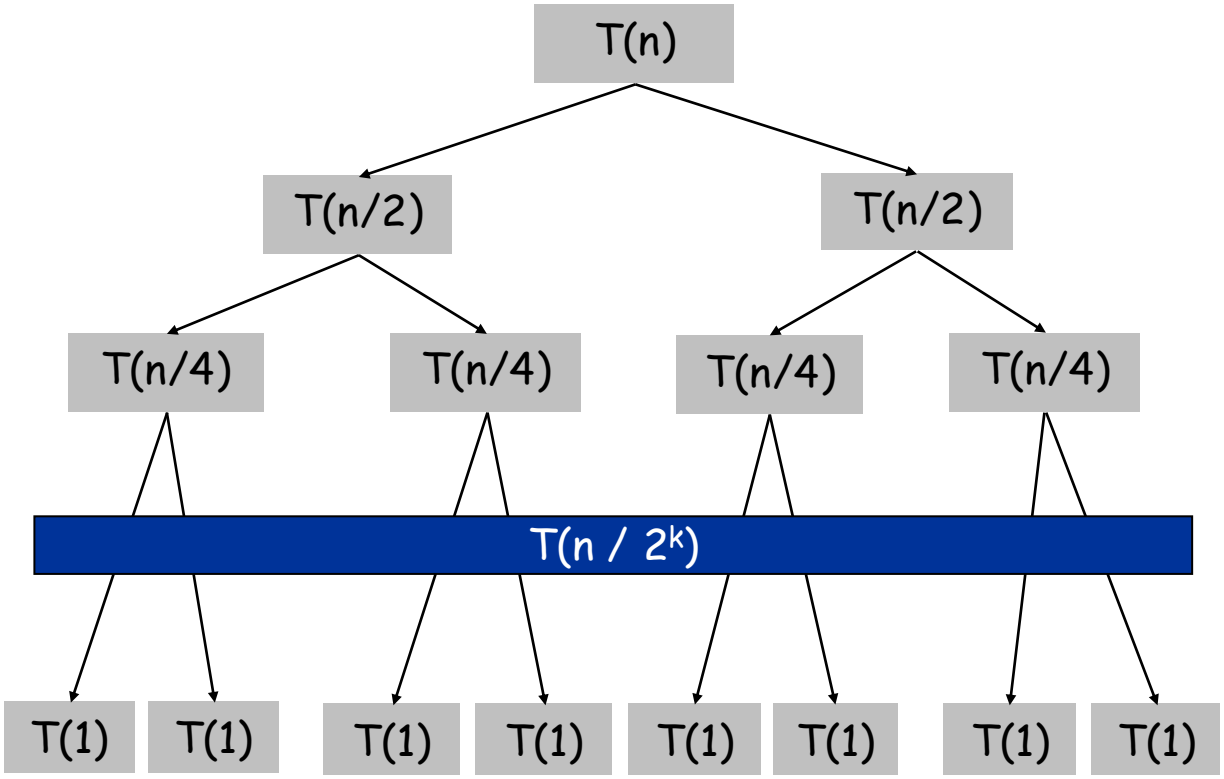
. . .

2^k (of size $n/2^k$)

. . .

n (of size 1)

Proof by unrolling: T(n) = 2 T(n/2) + O(n)



1 (of size n) → cn

2 (of size n/2) → cn

4 (of size n/4) → cn

$\log_2 n$

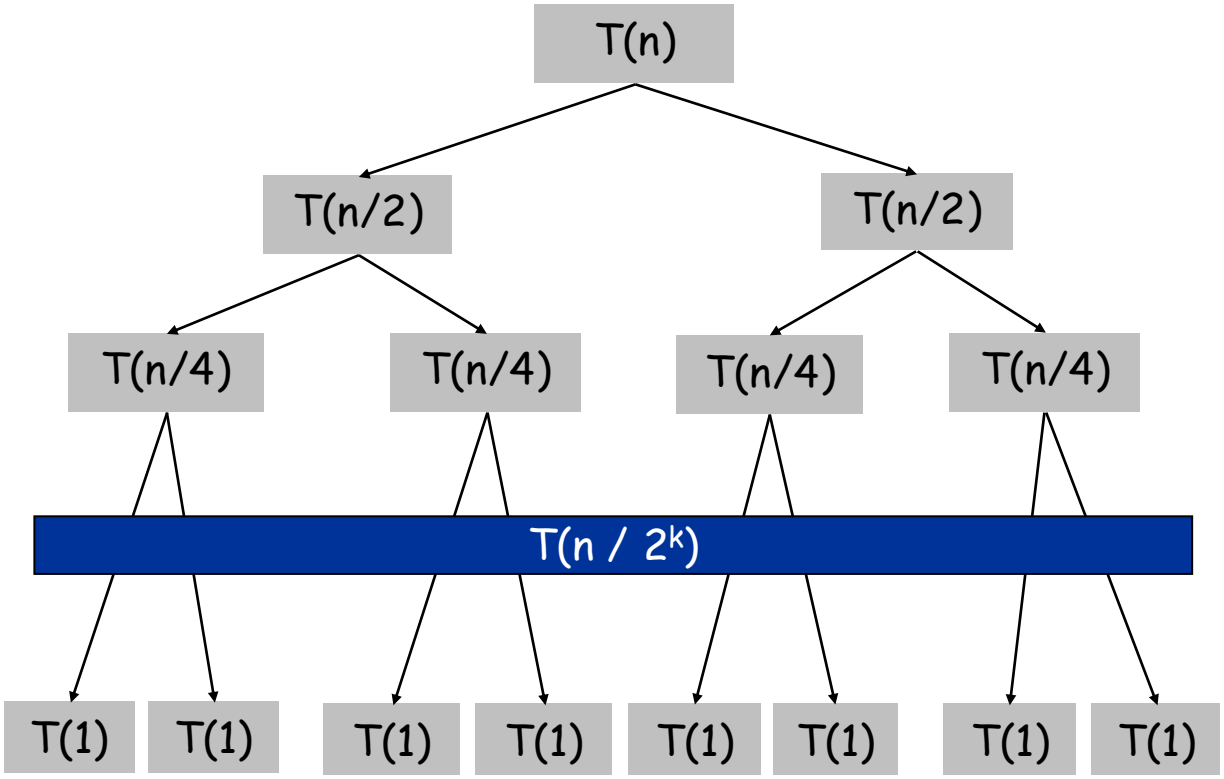
. . .

2^k (of size $n/2^k$) → cn

. . .

n (of size 1) → cn

Proof by unrolling: T(n) = 2 T(n/2) + O(n)



1 (of size n) → cn

2 (of size n/2) → cn

4 (of size n/4) → cn

log n

. . .

2^k (of size $n/2^k$) → cn

. . .

n (of size 1) → cn

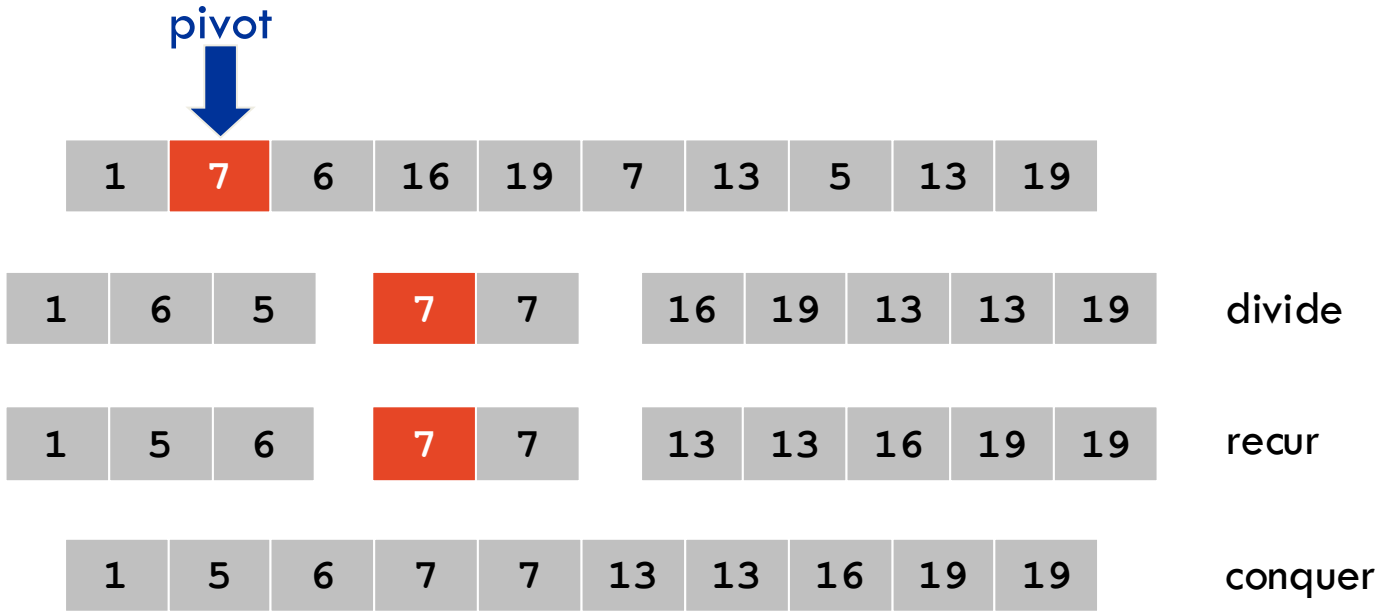
cn log n

Some recurrence formulas with solutions

Recurrence	Solution
$T(n) = 2\,T(n/2) + O(n)$	$T(n) = O(n \log n)$
$T(n) = 2\,T(n/2) + O(\log n)$	$T(n) = O(n)$
$T(n) = 2\,T(n/2) + O(1)$	$T(n) = O(n)$
$T(n) = T(n/2) + O(n)$	$T(n) = O(n)$
$T(n) = T(n/2) + O(1)$	$T(n) = O(\log n)$
$T(n) = T(n-1) + O(n)$	$T(n) = O(n^2)$
$T(n) = T(n-1) + O(1)$	$T(n) = O(n)$

Quick sort

1. Divide Choose a random element from the list as the pivot
   Partition the elements into 3 lists:
   (i) less than, (ii) equal to and (iii) greater than the pivot
2. Recur Recursively sort the less than and greater than lists
3. Conquer Join the sorted 3 lists together



Quick sort complexity analysis

Divide step (pick pivot and split) takes $O(n)$

Recur step (solve left and right subproblem) takes $T(n_L) + T(n_R)$

Conquer step (merge subarrays) takes $O(n)$

Now we can set up the recurrence for $T(n)$:

$$E[T(n)] = \begin{cases} E[T(n_L) + T(n_R)] + O(n) & \text{for } n > 1 \\ O(1) & \text{for } n = 1 \end{cases}$$

This solves to $E[T(n)] = O(n \log n)$ expected time

(details available on the textbook but not examinable)

Interlude: Comparison sorting lower bound

So far we've seen many sorting algorithms. Some run in $O(n^2)$ time while others run in $O(n \log n)$ time.

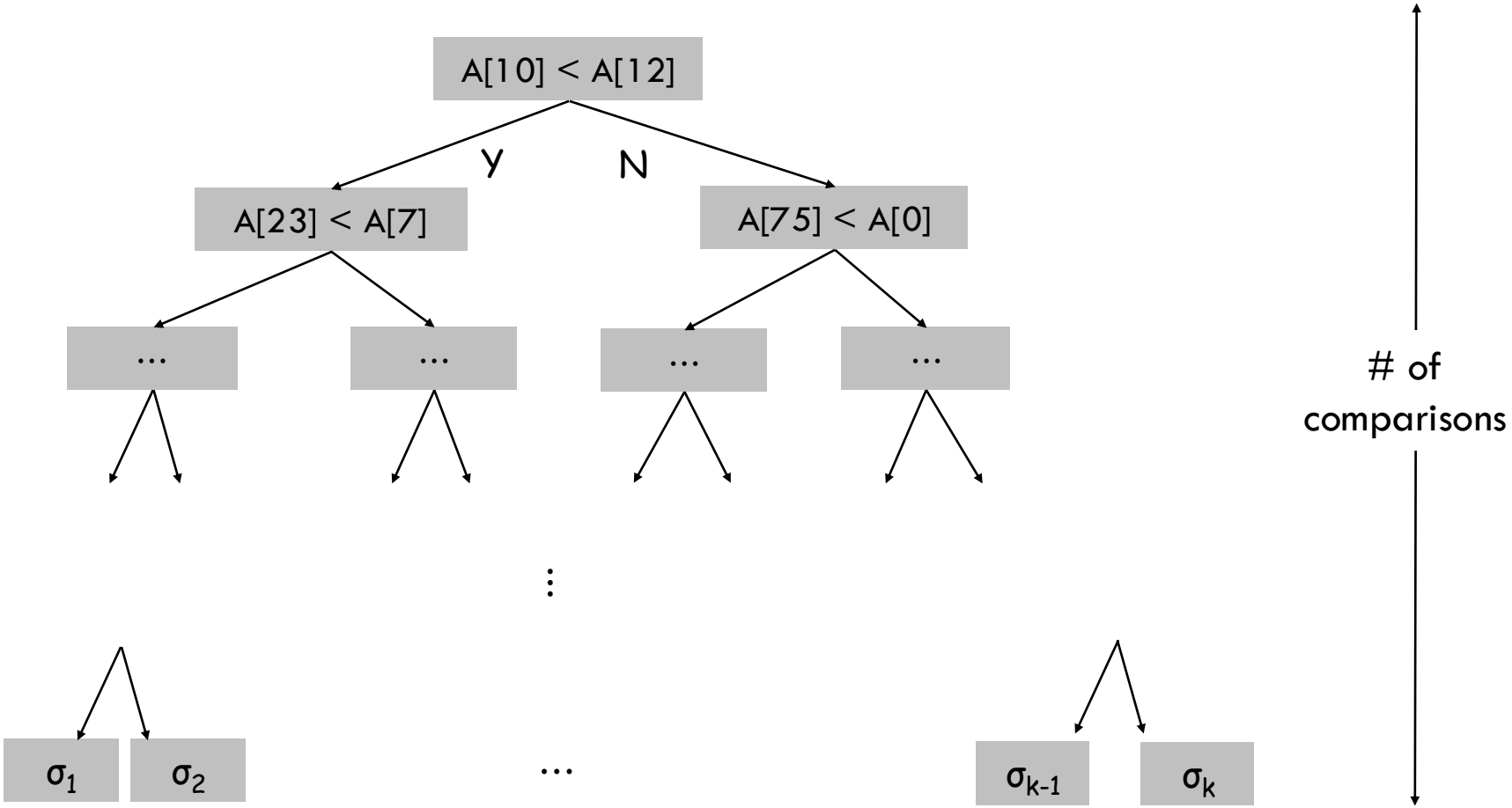
These algorithms work by performing pair-wise comparisons between elements of the sequence we are trying to sort

Such algorithms can be viewed as a decision tree where:

- each internal node compares two indices of the input array
- each external node corresponds to a permutation of $\{1, \ldots, n\}$

The height of the decision tree is a lower bound on the running time of the algorithm, since it only counts number of comparisons

Decision tree



The output of a leaf is A[σ(1)], A[σ(2)], … , A[σ(n)]

Interlude: Comparison sorting lower bound

Fact: Comparison-based sorting algorithms take $\Omega(n \log n)$ time

Proof:

The decision tree associated with a comparison-based sorting algorithm is binary and has $n!$ external nodes. Thus the height is $\log n!$ which is $\Omega(n \log n)$

$$\begin{aligned}\log n! &= \log (n * (n-1) * \ldots * 1) \\ &= \log n + \log(n-1) + \ldots + \log 1 \\ &> n/2 * (\log n/2) \\ &= \Omega(n \log n)\end{aligned}$$

Remember

Important:

Simply using Merge-Sort in your algorithm doesn't make your algorithm a divide and conquer algorithm.

Example:

A greedy algorithm first sorts the input in some way and then processes the items one by one in that order. Using Merge-Sort for the sorting step doesn't change the fact that the algorithm computes the solution in a greedy way.